

DATA STRUCTURES AND ALGORITHM

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	BHUMA SANVIKA
ROLL NO	AV.SC.U4CSE25020
YEAR/SEM/SECTION	2nd YEAR /SEM-3/CSE A
CREDITS	05

SUBMITTED TO	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFFESSOR (Sl. Grade)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	TITLE	PAGE NO	DATE
1.	**WEEK-1**:1a. Recursive countdown from n to Launch. 1b. Recursive calculation of compound growth factor. 1c. Recursive search for employee ID in list. 1d. Recursive factorial calculation for parcel arrangements. 1e. Recursive Fibonacci series generation.	4-10	23-07-26
2.	**WEEK-2**: 2a. Python program to perform linear search on the given input. 2b. Python program to perform binary search on the given input. 2c. Verify by taking sorted array and unsorted array as an input.	10-14	30-07-26
3.	WEEK -3: 3a. Python program to perform bubble sort on the given input. 3b. Python program to perform insertion sort on the given input. 3c. Python program to perform selection sort on the given input. 3d. Python program to perform merge sort on the given input. 3e. Python program to perform quick sort on the given input.	14-23	06-08-26
4.	WEEK-4: 4a. Write the python program to implement a single linked list which could perform the below operations,by taking an input from the user. 4b. Write the python program to implement a double linked list which could perform the below operations,by taking an input from the user. 4c. Write the python program to implement a circular linked list which could perform the below operations,by taking an input from the user.	23-54	13-08-26

5.	WEEK-5:Write a python program to implement a stack using an Array, Linked list. Implement the stack operations such as push, pop, peak and also display the stack elements	54-62	27-08-26
6.	WEEK-6: 6a. Write a python program to implement queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach implement using array, linked list. 6b. Write a python program to implement queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach implement using circular linked list.	62-78	03-09-26
7.	WEEK-7: 7. Write a program to implement a binary tree using an array and linked list with all of its operation.	78-85	24-09-26
8.	WEEK-8: 8. Write a python program to implement Binary Search Trees with its operations.	86-90	08-10-26
9.	WEEK-9: 9. Write a python program to implement AVL trees with its operation.	90-95	22-10-26
10.	WEEK-10: 10.Write a python program to Implement Hashing Using Separate Chaining, Linear Probing and Quadratic Probing.	95-99	29-10-26

LAB -1

1)a.AIM: A rocket launch system displays a countdown before launch. Instead of using a loop implement the countdown using recursion .write a recursion python program to display a countdown from n to 1 followed by “launch”.

PROGRAM:

```
def launch(n):
    for i in range(n,-1,-1):

        if i==0:
            print("launch")

        else:
            print(i)

n=int(input("enter no. of seconds to launch:"))
launch(n)
```

OUTPUT:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

==================== RESTART: C:/Users/haisr/Desktop/lab1.py ==================
enter no. of seconds to launch:5
5
4
3
2
1
launch
```

EXPLANATION:

This program displays a countdown from n to 1 and finally prints 'launch'. Although the aim mentions recursion, the given code uses a loop.

ERROR TABLE:

ERROR	CAUSE	SOLUTION
Indentation Error	Incorrect indentation	Align blocks properly
Name Error	Variable not defined	Define before use
Value Error	Non-integer input	Enter valid integer

1.bAIM: banks calculate the growth of an investment using compoundinterest the formula requires calculating powers. Which can be implemented recursively. Write a recursive program to calculate p^n(power),where p=principal growth factor, n= number of years.

PROGRAM:

```
def intrest(p,n):
    if n==0:
        return 1
    else:
        return p*intrest(p,n-1)
n=int(input("enter no.of years:"))
p=int(input("enter a number:"))
res=intrest(p,n)
print(res)
```

OUTPUT:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

========= RESTART: C:/Users/haisr/Desktop/my python/my python/lab1.2.py ========
enter no.of years:1
enter a number:2
2
|
```

EXPLANATION:This program recursively calculates p^n by multiplying p until n becomes 0.

ERROR TABLE:

ERROR	CAUSE	SOLUTION
Recursion error	Very large value of n	Reduce input

Name Error	Wrong function/variable name	Check spelling
Value Error	Non-integer input	Enter valid integer

1.cAIM: An HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID .write a recursive function to search for an employee ID in a list.

PROGRAM:

```
def search(m):


    key=m
    for i in range(n-1):
        if arr[i]==key:
            return i


n=int(input("enter no.of employees to add:"))
arr=[]
for i in range(n):
    a=int(input("enter a value:"))
    arr.append(a)
print(arr)
m=int(input("enter a element to search:"))
result=search(m)
if result==-1:
     print("element is not found")
else:
     print("employee id is stored at the index:",result)
```

OUTPUT:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

========= RESTART: C:/Users/haisr/Desktop/my python/my python/lab1.3.py ========
enter no.of employees to add:2
enter a value:10
enter a value:11
[10, 11]
enter a element to search:10
employee id is stored at the index: 0
```

EXPLANATION:

This program searches for an employee ID in a list and returns its index if found.

ERROR TABLE:

ERROR	CAUSE	SOLUTION
Syntax error	Wrong quotes/brackets	Correct syntax
Logic error	Returns None instead of -1	Return -1 when not found

1.d.AIM: In a logistics company the number of possible ways to arrange n different parcels is calculated using the factorial formula(n!). develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
def fact(n):
    fact=1
    for i in range(1,n+1):
        fact=fact*i


    return fact
n=int(input("enter a number to find its factorial:"))
result=fact(n)
print(result)
```

OUTPUT:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

========= RESTART: C:/Users/haisr/Desktop/my python/my python/lab1.4.py ========
enter a number to find its factorial:3
6
```

EXPLANATION:This program calculates the factorial of a number using iteration (not recursion).

ERROR TABLE:

ERROR	CAUSE	SOLUTION
Overflow/large input	Large input value	Give smaller input
Value error	Invalid input	Even integer

1.e.AIM: write a recursive python program to print the first n terms of the Fibonacci series.

PROGRAM:

```
def fib(n):
    if n==0:
        return 0
    elif n==1:
        return 1
    else:
        return fib(n-1)+fib(n-2)


n=int(input("enter no of terms:"))
if n<=-1:
    print("invalid input")
else:
    for i in range(n):
        print(fib(i),end=" ")
```

OUTPUT:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

========= RESTART: C:/Users/haisr/Desktop/my python/my python/lab1.5.py ========
enter no of terms:3
0 1 1
```

EXPLANATION:

The program uses recursion to generate Fibonacci numbers. fib (0)=0 and fib (1) =1 are the base cases. For every other term, fib(n)=fib(n-1) + fib(n-2). The loop prints the first n terms.

ERROR TABLE:

ERROR	CAUSE	SOLUTION
Recursion error	Large value of n	Use iteration
Invalid input	Negative value of n	Validate input
Slow execution	Repeated recursive calls	Use dynamic programming

PROGRAM	TIME COMPLEXITY	SPACE COMPLEXITY	REASON
1.countdown to launch	**O(n)**	**O(1)**	The program uses a loop that prints from **n to 0** once.
2.power calculation	**O(n)**	**O(n)**	One recursive call is made for each value of **n**. The recursion stack stores **n** calls.
3.employee ID search	**O(n)**	**O(1)**	The program performs a linear search through the list.
4.Factorial	**O(n)**	**O(1)**	The program calculates factorial using a loop from **1 to n**.
	$O(2^n)$	**O(n)**	

5.Fibonacci series			Each function call creates two more recursive calls, leading to exponential time. The recursion stack reaches depth **n**.

LAB-2

2.a.AIM: Write a program to perform linear search on a given input.

PROGRAM:

```
def linear_search(arr,key):
    for i in range(len(arr)):
        if arr[i]==key:
            return i
    return -1

n = int(input("Enter no. of elements:"))
arr=[]
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
key=int(input("Enter element to search:"))
result=linear_search(arr,key)
if result!=-1:
    print("Element found at index",result)
else:
    print("Element not found.")
```

OUTPUT:

```
Python 3.11.4 (tags/v3.11.4:d2340ef, Jun  7 2023, 05:45:37) [MSC v.1934 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

= RESTART: C:/Users/Lab/Desktop/cse25020/lab2.py
Enter no. of elements:3
Enter the elements:
12
2
3
Enter element to search:3
Element found at index 2
```

EXPLANATION:

- The linear_search() function checks each element of the list one by one and compares it with the given `key`.
- If the key is found, its index is returned; otherwise, the function returns `-1`.
- The program takes the array and search element as input and displays whether the element is found.

ERROR TABLE:

S.no.	Errors	Rectification
1.	ValueError occurs if a non-integer value is entered when int(input()) is used.	Enter only integer values for the number of elements and array elements.
2.	TypeError may occur if len(arr) is used on a variable that is not a list.	Make sure arr is initialized as a list using arr = [] and elements are appended to it.

2.b.AIM: Write a Python program to perform binary search on the given input. Verify by taking sorted array and unsorted array as input.

PROGRAM:

```
def binary_search(arr,key):
    low=0
    high=len(arr)-1
    while low<=high:
        mid=(low+high)//2
        if arr[mid]==key:
            return mid
        elif arr[mid]<key:
```

```
            low=mid+1
        else:
            high=mid-1
    return -1


n = int(input("Enter no. of elements:"))
arr=[]
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))


key=int(input("Enter element to search:"))
result=binary_search(arr,key)
if result!=-1:
    print("Element found at index",result)
else:
    print("Element not found.")
```

OUTPUT:

```
Python 3.11.4 (tags/v3.11.4:d2340ef, Jun  7 2023, 05:45:37) [MSC v.1934 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

= RESTART: C:/Users/Lab/Desktop/cse25020/lab2.2.py
Enter no. of elements:3
Enter the elements:
1
2
3
Enter element to search:2
Element found at index 1
```

2.c.PROGRAM:

```
def binary_search(arr,key):
    low=0
    high=len(arr)-1
```

```
    while low<=high:
        mid=(low+high)//2
        if arr[mid]==key:
            return mid
        elif arr[mid]<key:
            low=mid+1
        else:
            high=mid-1
    return -1


n = int(input("Enter no. of elements:"))
arr=[]
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
 if arr==sorted(arr):
    print("\n The input list is already sorted.")
else:
    print("\n The input list is not sorted.")
    print("Sorting the list...")
    arr.sort()
print("Sorted list.",arr)
 key=int(input("Enter element to search:"))
result=binary_search(arr,key)
if result!=-1:
    print("Element found at index",result)
else:
    print("Element not found.")
```

OUTPUT:

```
Python 3.11.4 (tags/v3.11.4:d2340ef, Jun  7 2023, 05:45:37) [MSC v.1934 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

= RESTART: C:/Users/Lab/Desktop/cse25020/lab2.3.py
Enter no. of elements: 3
Enter the elements:
1
2
3

The input list is already sorted.
Sorted list: [1, 2, 3]
Enter element to search: 2
Element found at index 1
```

EXPLANATION:

- The binary_search() function repeatedly divides the sorted list into two halves and compares the middle element with the search key.
- If the middle element is smaller than the key, the search continues in the right half; otherwise, it searches the left half.
- The second program verifies whether the input is sorted and sorts it using arr.sort() if necessary before performing binary search.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Binary search may give an incorrect result if the input array is unsorted.	Check whether the array is sorted and use arr.sort() before calling binary_search().
2.	IndexError can occur if mid is calculated or the low/high boundaries are modified incorrectly.	Initialize low = 0, high = len(arr) - 1 and update them as low = mid + 1 or high = mid - 1.

LAB-3

AIM: 3a. Write a python program to sort the elements using a bubble sort technique. Take the input elements from the user.

Program:

```
n = int(input("Enter the number of elements: "))

arr = []

print("Enter the elements:")

for i in range(n):

arr.append(int(input()))
```

```
for i in range(n - 1):
for j in range(n - i - 1):
if arr[j] < arr[j + 1]:
arr[j], arr[j + 1] = arr[j + 1], arr[j]
print("Elements after Bubble Sort:")
print(arr)
```

Output:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

==================== RESTART: C:/Users/haisr/Pictures/AND.PY ====================
Enter the number of elements: 3
Enter the elements:
1
2
3
Elements after Bubble Sort:
[3, 2, 1]
```

Explanation:

• n = int(input(...)) – Gets the number of elements.

• arr = [] – Creates an empty list to store elements.

• for i in range(n) – Takes n elements as input.

• arr.append(...) – Adds each element to arr.

• for i in range(n - 1) – Performs Bubble Sort passes.

• for j in range(n - i - 1) – Compares adjacent elements.

• if arr[j] < arr[j + 1] – Checks if the current element is smaller.

• arr[j], arr[j + 1] = ... – Swaps the elements to sort in descending order.

• print(arr) – Displays the sorted list.

Error Table:

S.NO	Error Obtained	Rectification
1.	Sorting Order Error – Using > instead of < in the if condition changes the order to ascending.	Use if arr[j] < arr[j + 1]: for descending order.

AIM: 3b.Write a python program to sort the elements using a insertion sort technique. Take the input elements from the user.

Program:

```
def insertion_sort(arr):
n = len(arr)
for i in range (1,n):
key = arr[i]
j = i - 1
while j >= 0 and arr[j] > key:
arr[j+1] = arr[j]
j = j - 1
arr[j+1] = key
return arr
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
element = int(input())
arr.append(element)
result = insertion_sort(arr)
print("Elements after Insertion Sort:")
print(result)
```

Output:



Explanation:

• n = len(arr) – Gets the list size.

• for i in range(1, n) – Selects elements from the second position.

• key = arr[i] – Stores the current element.

• j = i - 1 – Sets the previous index.

• while j >= 0 and arr[j] > key – Compares previous elements with key.

• arr[j + 1] = arr[j] – Shifts larger elements right.

• arr[j + 1] = key – Inserts key in the correct position.

• return arr – Returns the sorted list.

• result = insertion_sort(arr) – Calls the function.

• print(result) – Displays the sorted list.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Insertion Error – Placing key at the wrong index.	Use arr[j + 1] = key after shifting elements.
2.	Function Call Error – Not calling insertion_sort(arr) correctly.	Pass the input list arr to the function.

AIM: 3c.Write a python program to sort the elements using a selection sort technique. Take the input elements from the user.

Program:

```
def selection_sort(arr):
n=len(arr)
for i in range(n-1):
min_index=i
for j in range (i+1,n):
if arr[j]<arr[min_index]:
min_index=j
arr[i],arr[min_index]=arr[min_index],arr[i]
return arr
n=int(input("Enter the number of elements: "))
arr=[]
print("Enter the elements:")
for i in range(n):
element=int(input())
arr.append(element)
result=selection_sort(arr)
print("Elements after Selection Sort:")
print(result)
```

Output:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

========== RESTART: C:/Users/haisr/Pictures/fgh.py ==========
Enter the number of elements: 3
Enter the elements:
1
2
3
Elements after Selection Sort:
[1, 2, 3]
```

Explanation:

• def selection_sort(arr) – Defines the Selection Sort function.

• n = len(arr) – Gets the number of elements in arr.

• for i in range(n-1) – Performs sorting passes.

• min_index = i – Assumes the first element is the smallest.

• for j in range(i+1,n) – Checks the remaining elements.

• if arr[j] < arr[min_index] – Finds the smallest element.

• arr[i], arr[min_index] = ... – Swaps the smallest element into position.

• return arr – Returns the sorted list.

• result = selection_sort(arr) – Calls the function.

• print(result) – Displays the sorted list in ascending order.

Error Table:

S.NO	Error Obtained	Rectification
1.	Swap Error – Incorrectly swapping the elements.	Use arr[i], arr[min_index] = arr[min_index], arr[i].
2.	Minimum Index Error – Incorrectly assigning min_index.	Use min_index = i.

AIM: 3d.Write a python program to sort the elements using a merge sort technique. Take the input elements from the user.

Program:

```
def merge_sort(arr):
if len(arr) <= 1:
return arr
mid=len(arr)//2
left=merge_sort(arr[:mid])
right=merge_sort(arr[mid:])
i = j = k = 0
while i < len(left) and j < len(right):
```

```
if left[i] <= right[j]:
arr[k] = left[i]
i += 1
else:
arr[k] = right[j]
j += 1
k += 1
while i < len(left):
arr[k] = left[i]
i += 1
k += 1
while j < len(right):
arr[k] = right[j]
j += 1
k += 1
return arr
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
element = int(input())
arr.append(element)
result = merge_sort(arr)
print("Elements after Merge Sort:")
print(result)
```

Output:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

========== RESTART: C:/Users/haisr/Pictures/abd.py ==========
Enter the number of elements: 3
Enter the elements:
1
2
3
Elements after Merge Sort:
[1, 2, 3]
```

Explanation:

• def merge_sort(arr) – Defines the Merge Sort function.

• if len(arr) <= 1 – Stops when the list has one or no elements.

• mid = len(arr)//2 – Finds the middle of the list.

• left and right – Divide the list into two halves recursively.

• i = j = k = 0 – Initializes the indexes.

• while loops – Compare and merge the elements.

• if left[i] <= right[j] – Selects the smaller element.

• arr[k] – Places the selected element in the list.

Error Table:

S.NO	Error Obtained	Rectification
1.	Index Error – Incorrectly updating i, j, or k.	Increment the appropriate index after each insertion.
2.	Recursion Error – Incorrectly dividing the list.	Use arr[:mid] and arr[mid:].

AIM: 3e.Write a python program to sort the elements using a quick sort technique. Take the input elements from the user.

Program:

```
def quicksort(a,low,high):
```

```
if low < high:
i = low
j = high
pivot = low
while i < j:
while i < len(a) and a[i]<=a[pivot]:
i +=1
while a[j] > a[pivot]:
j = j -1
if i < j:
a[i],a[j] = a[j],a[i]
a[j],a[pivot] = a[pivot],a[j]
quicksort(a,low,j-1)
quicksort(a,j+1,high)
a = list(map(int,input("Enter numbers to sort:").split(" ")))
n = len(a)
quicksort(a,0,n-1)
print("Sorted array:")
print(a)
```

Output:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

== RESTART: C:/Users/haisr/AppData/Local/Programs/Python/Python313/sanvi.py.py =
Enter numbers to sort:3
Sorted array:
[3]
```

Explanation:

• def quicksort(a, low, high) – Defines the Quick Sort function.

• if low < high – Checks the sorting range.

• i and j – Set left and right positions.

• pivot = low – Selects the first element as pivot.

• while i < j – Performs partitioning.

• Inner while loops – Find elements for swapping.

• a[i], a[j] = ... – Swaps elements.

• a[j], a[pivot] = ... – Places the pivot correctly.

• quicksort() calls – Sort the two partitions.

• input().split() – Takes space-separated numbers.

• print(a) – Displays the sorted array.

Error Table:

S.NO	Error Obtained	Rectification
1.	Pivot Error – Incorrectly selecting the pivot index.	Use pivot = low.
2.	Swap Error – Incorrect swapping of a[i] and a[j].	Use a[i], a[j] = a[j], a[i].

LAB-4

4.a. AIM: Write a python program to implement a single linked list which would perform the below operations, by taking an input from the user.

1. Create a linked list. 2. Insert at beginning. 3. Insert at end.

4. Insert at a specific index. 5. Delete by value. 6. Delete 1st node. 7. Delete last node.
8. Count no. of nodes 9. Display.

10. Exit.

Programme:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
class LinkedList:
    def __init__(self):
        self.head = None

    def insert_begin(self, data):
        new = Node(data)
        new.next = self.head
        self.head = new

    def insert_end(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new

    def deleteAtBeg(self):
        if self.head is None:
            print("No Data to delete")
        else:
            temp = self.head
            self.head = temp.next
            print("Deleted Value =", temp.data)
```

```
def deleteAtEnd(self):
    if self.head is None:
        print("No Data to delete")

    elif self.head.next is None:
        print("Deleted Value =", self.head.data)
        self.head = None

    else:
        temp = self.head

        while temp.next.next:
            temp = temp.next

        print("Deleted Value =", temp.next.data)
        temp.next = None

def delete(self, value):
    if self.head is None:
        print("No Data to delete")
        return

    temp = self.head

    if temp.data == value:
        self.head = temp.next
        print("Value deleted")
        return

    while temp.next and temp.next.data != value:
```

```
            temp = temp.next

        if temp.next is None:
            print("Value not present")
        else:
            temp.next = temp.next.next
            print("Value deleted")

    def count(self):
        if self.head is None:
            print("No Linked List")
        else:
            c = 0
            temp = self.head

            while temp:
                c += 1
                temp = temp.next

            print(f"Number of nodes = {c}")

    def display(self):
        if self.head is None:
            print("No Linked List")
        else:
            temp = self.head

            while temp:
                print(temp.data, end=" -> ")
                temp = temp.next
```

```
            print("None")

ll = LinkedList()

while True:
    print("\n--- Singly Linked List ---")
    print("1. Insert at Beginning")
    print("2. Insert at End")
    print("3. Delete at Beginning")
    print("4. Delete at End")
    print("5. Delete a Value")
    print("6. Count Nodes")
    print("7. Display")
    print("8. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter data: "))
        ll.insert_begin(data)

    elif choice == 2:
        data = int(input("Enter data: "))
        ll.insert_end(data)

    elif choice == 3:
        ll.deleteAtBeg()

    elif choice == 4:
```

```
            ll.deleteAtEnd()

        elif choice == 5:
            value = int(input("Enter value to delete: "))
            ll.delete(value)

        elif choice == 6:
            ll.count()

        elif choice == 7:
            ll.display()

        elif choice == 8:
            print("Program ended")
            break

        else:
            print("Invalid choice")
```

output:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

================== RESTART: C:/Users/haisr/Downloads/DSA.4.2.PY ==================

--- Singly Linked List ---
1. Insert at Beginning
2. Insert at End
3. Delete at Beginning
4. Delete at End
5. Delete a Value
6. Count Nodes
7. Display
8. Exit
Enter your choice: 1
Enter data: 2

--- Singly Linked List ---
1. Insert at Beginning
2. Insert at End
3. Delete at Beginning
4. Delete at End
5. Delete a Value
6. Count Nodes
7. Display
8. Exit
Enter your choice: 1
Enter data: 3

--- Singly Linked List ---
1. Insert at Beginning
2. Insert at End
3. Delete at Beginning
4. Delete at End
5. Delete a Value
6. Count Nodes
7. Display
8. Exit
Enter your choice: 7
```

```
8. Exit
Enter your choice: 1
Enter data: 2

--- Singly Linked List ---
1. Insert at Beginning
2. Insert at End
3. Delete at Beginning
4. Delete at End
5. Delete a Value
6. Count Nodes
7. Display
8. Exit
Enter your choice: 1
Enter data: 3

--- Singly Linked List ---
1. Insert at Beginning
2. Insert at End
3. Delete at Beginning
4. Delete at End
5. Delete a Value
6. Count Nodes
7. Display
8. Exit
Enter your choice: 7
3 -> 2 -> None

--- Singly Linked List ---
1. Insert at Beginning
2. Insert at End
3. Delete at Beginning
4. Delete at End
5. Delete a Value
6. Count Nodes
7. Display
8. Exit
Enter your choice: 8
Program ended
```

Explanation:

1)Class Definitions:

1. Node class o Represents a single node in the linked list. o Each node has:

▪ data → stores the value.

▪ next → pointer to the next node (initially None).

2. SLL class o Manages the linked list operations. Has a head pointer (initially None) that points to the first node.

2)Insert at Beginning:

• Creates a new node.

• Points its next to current head.

• Updates head to new node.

3)Insert at End

• Creates a new node.

• If list is empty → new node becomes head.

• Otherwise → traverses to last node and attaches new node.

4) Insert at Index

• Asks for index and data.

• If index = 0 → behaves like "insert at beginning".

• Otherwise → traverses to (index-1) node and inserts new node in between.

5) Delete from Beginning

• Moves head to head.next.

• Effectively removes the first node.

6) Delete from End

• If list empty → prints "List is empty".

• If only one node → sets head = None.

• Otherwise → traverses to second-last node and sets its

next= None.

7) Delete from Index

• Asks for index.

• If index = 0 → removes head.

• Otherwise → traverses to (index-1) node and skips the next node.

8) Count Nodes

• Traverses through list.

• Counts nodes until None.

9) Display

• Traverses from head to end.

• Prints each node’s data in data -> data -> None format.

Error table:

sno	error	rectification

1.	new.next = None	new.next = self.head
2.	Logical error in insertion if index is negative.	Validate index: if index < 0: print ("Invalid index").
3.	temp.next = temp.next.next	temp.next = None *(when deleting the last node)*

4.b.Aim: Write a python program to implement a doubly linked list which would perform the below operations, by taking an input from the user.

1. Create a linked list. 2. Insert at beginning. 3. Insert at end.

4. Insert at a specific index. 5. Delete by value. 6. Delete 1st node. 7. Delete last node. 8. Count no. of nodes 9. Display.

10. Exit.

Programme:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def insert_begin(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
```

```
        else:
            new_node.next = self.head
            self.head.prev = new_node
            self.head = new_node

    def insert_end(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            temp = self.head

            while temp.next is not None:
                temp = temp.next

            temp.next = new_node
            new_node.prev = temp

    def insert_at_position(self, data, position):
        new_node = Node(data)

        if position == 1:
            self.insert_begin(data)
            return

        temp = self.head
        count = 1

        while temp is not None and count < position - 1:
```

```
            temp = temp.next
            count += 1

        if temp is None:
            print("Invalid position")
            return

        new_node.next = temp.next
        new_node.prev = temp

        if temp.next is not None:
            temp.next.prev = new_node

        temp.next = new_node

    def delete_begin(self):
        if self.head is None:
            print("List is empty")
            return

        self.head = self.head.next

        if self.head is not None:
            self.head.prev = None

    def delete_end(self):
        if self.head is None:
            print("List is empty")
            return
```

```
        temp = self.head

        if temp.next is None:
            self.head = None
            return

        while temp.next is not None:
            temp = temp.next

        temp.prev.next = None

    def delete_at_position(self, position):
        if self.head is None:
            print("List is empty")
            return

        if position == 1:
            self.delete_begin()
            return

        temp = self.head
        count = 1

        while temp is not None and count < position:
            temp = temp.next
            count += 1

        if temp is None:
            print("Invalid position")
            return
```

```
        if temp.next is not None:
            temp.next.prev = temp.prev

        if temp.prev is not None:
            temp.prev.next = temp.next

    def count(self):
        temp = self.head
        count = 0

        while temp is not None:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)

    def display(self):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head

        print("Doubly Linked List:", end=" ")

        while temp is not None:
            print(temp.data, end=" <-> ")
            temp = temp.next
```

```
        print("END")

dll = DoublyLinkedList()
while True:
    print("\n--- DOUBLY LINKED LIST ---")
    print("1. Insert Begin")
    print("2. Insert End")
    print("3. Insert at Specific Position")
    print("4. Delete Begin")
    print("5. Delete End")
    print("6. Delete at Specific Position")
    print("7. Count")
    print("8. Display")
    print("9. End")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter data: "))
        dll.insert_begin(data)

    elif choice == 2:
        data = int(input("Enter data: "))
        dll.insert_end(data)

    elif choice == 3:
        data = int(input("Enter data: "))
        position = int(input("Enter position: "))
        dll.insert_at_position(data, position)
```

```
    elif choice == 4:
        dll.delete_begin()

    elif choice == 5:
        dll.delete_end()

    elif choice == 6:
        position = int(input("Enter position: "))
        dll.delete_at_position(position)

    elif choice == 7:
        dll.count()

    elif choice == 8:
        dll.display()

    elif choice == 9:
        print("Program ended")
        break

    else:
        print("Invalid choice")
```

output:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

================== RESTART: C:/Users/haisr/Downloads/DSA.4.3.py ==================

--- DOUBLY LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 1
Enter data: 2

--- DOUBLY LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 4

--- DOUBLY LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
```

```
Enter data: 2

--- DOUBLY LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 4

--- DOUBLY LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 8
List is empty

--- DOUBLY LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 9
```

Explanation:

- Node has 3 parts:
    - data → stores value
    - prev → points to previous node
    - next → points to next node
- Doubly Linked List:
    - Each node can move in both directions.
    - First node's prev is None.
    - Last node's next is None.
    - Example: None ← 10 ↔ 20 ↔ 30 → None

Operations

1. Insert at Beginning
    - Create a new node.
    - New node points to the current head.
    - Current head's prev points to new node.
    - New node becomes the head.
2. Insert at End
    - Traverse to the last node.
    - Last node's next points to new node.
    - New node's prev points to the last node.
3. Delete from Beginning
    - Move head to the next node.
    - Set new head's prev to None.
    - If only one node exists, list becomes empty.
4. Delete from End
    - Traverse to the last node.
    - Set the previous node's next to None.
    - If only one node exists, list becomes empty.
5. Display Forward
    - Start from head.
    - Move using next.
    - Continue until None.
6. Display Backward
    - First move to the last node.
    - Move backward using prev.
    - Continue until None.
7. Main Program
    - Create a doubly linked list.
    - Display the menu.
    - Take user's choice.

- Perform the selected operation.
- Choice 7 exits the program.

Error table:

sno	error	rectification
1.	new_node.prev = None	new_node.prev = temp
2.	new_node.next = None	new_node.next = self.head
3.	self.head.prev = None	self.head.prev = new_node

4.c.Aim: Write a python program to implement a circular linked list which would perform the below operations, by taking an input from the user.

1. Create a linked list. 2. Insert at beginning. 3. Insert at end.

4. Insert at a specific index. 5. Delete by value. 6. Delete 1st node. 7. Delete last node.
8. Count no. of nodes 9. Display.

10. Exit.

Programme:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularLinkedList:
    def __init__(self):
        self.head = None

    # Insert at beginning
    def insert_begin(self, data):
        new_node = Node(data)
```

```
    if self.head is None:
        self.head = new_node
        new_node.next = self.head
    else:
        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        new_node.next = self.head
        temp.next = new_node
        self.head = new_node

# Insert at end
def insert_end(self, data):
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        new_node.next = self.head
    else:
        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        temp.next = new_node
        new_node.next = self.head
```

```
# Insert at specific position
def insert_at_position(self, data, position):
    if position == 1:
        self.insert_begin(data)
        return

    new_node = Node(data)
    temp = self.head
    count = 1

    while temp.next != self.head and count < position - 1:
        temp = temp.next
        count += 1

    if count != position - 1:
        print("Invalid position")
        return

    new_node.next = temp.next
    temp.next = new_node

# Delete beginning
def delete_begin(self):
    if self.head is None:
        print("List is empty")
        return

    if self.head.next == self.head:
        self.head = None
        return
```

```
        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        self.head = self.head.next
        temp.next = self.head

    # Delete end
    def delete_end(self):
        if self.head is None:
            print("List is empty")
            return

        if self.head.next == self.head:
            self.head = None
            return

        temp = self.head

        while temp.next.next != self.head:
            temp = temp.next

        temp.next = self.head

    # Delete at specific position
    def delete_at_position(self, position):
        if self.head is None:
            print("List is empty")
```

```
            return

        if position == 1:
            self.delete_begin()
            return

        temp = self.head
        count = 1

        while temp.next != self.head and count < position - 1:
            temp = temp.next
            count += 1

        if temp.next == self.head:
            print("Invalid position")
            return

        temp.next = temp.next.next

    # Count
    def count(self):
        if self.head is None:
            print("Number of nodes: 0")
            return

        temp = self.head
        count = 0

        while True:
            count += 1
```

```
            temp = temp.next

            if temp == self.head:
                break

        print("Number of nodes:", count)

    # Display
    def display(self):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head

        print("Circular Linked List:", end=" ")

        while True:
            print(temp.data, end=" -> ")
            temp = temp.next

            if temp == self.head:
                break

        print("HEAD")


# Main program
cll = CircularLinkedList()
while True:
```

```
print("\n--- CIRCULAR LINKED LIST ---")
print("1. Insert Begin")
print("2. Insert End")
print("3. Insert at Specific Position")
print("4. Delete Begin")
print("5. Delete End")
print("6. Delete at Specific Position")
print("7. Count")
print("8. Display")
print("9. End")
choice = int(input("Enter your choice: "))

if choice == 1:
    data = int(input("Enter data: "))
    cll.insert_begin(data)

elif choice == 2:
    data = int(input("Enter data: "))
    cll.insert_end(data)

elif choice == 3:
    data = int(input("Enter data: "))
    position = int(input("Enter position: "))
    cll.insert_at_position(data, position)

elif choice == 4:
    cll.delete_begin()

elif choice == 5:
    cll.delete_end()
```

```
elif choice == 6:
    position = int(input("Enter position: "))
    cll.delete_at_position(position)

elif choice == 7:
    cll.count()

elif choice == 8:
    cll.display()

elif choice == 9:
    print("Program ended")
    break
else:
    print("Invalid choice")
```

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

========== RESTART: C:/Users/haisr/Downloads/DSA4.1.py ==========

--- CIRCULAR LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 1
Enter data: 1

--- CIRCULAR LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 5

--- CIRCULAR LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
```

```
Enter data: 1

--- CIRCULAR LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 5

--- CIRCULAR LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 8
List is empty

--- CIRCULAR LINKED LIST ---
1. Insert Begin
2. Insert End
3. Insert at Specific Position
4. Delete Begin
5. Delete End
6. Delete at Specific Position
7. Count
8. Display
9. End
Enter your choice: 9
Program ended
```

Explanation:

- Node contains two parts:
    - data → stores the value.
    - next → stores the reference of the next node.
- Circular Linked List:
    - Last node points back to the first node (head).
    - There is no None at the end.
    - Example: 10 → 20 → 30 → 10

Operations

1. Create
    - Creates n nodes.

- First node becomes head.
- Last node points to head.

2. Insert at Beginning
    - Create a new node.
    - New node points to current head.
    - Last node points to new node.
    - New node becomes the new head.
3. Insert at End
    - Traverse until the last node.
    - Last node points to the new node.
    - New node points to head.
4. Insert at Specific Index
    - Move to the node before the required index.
    - Connect the new node between two nodes.
    - Newnode.next = temp.next
    - temp.next = new.node
5. Delete by Value
    - Search for the required value.
    - If found, remove that node.
    - Previous node is connected to the next node.
    - Special cases: only node or deleting the head.
6. Delete First Node
    - Move head to the second node.
    - Last node is connected to the new head.
7. Delete Last Node
    - Find the second-last node.
    - Make its next point to head.
8. Count Nodes
    - Start from head.
    - Traverse until reaching head again.

- Count each node.

9. Display
    - Start from head.
    - Print each node.
    - Stop when head is reached again.

Error table:

S.NO	Error Obtained	Rectification
1.	Forgetting to connect the last node to head.	Set the last nodes's next = head.
2.	Deleting from an empty list	Check if self.head is None before deletion.
3.	Infinite loop during traversal.	Stop traversal when temp = = self.head.

PROGRAM	TIME COMPLEXITY	SPACE COMPLEXITY	REASON
1. Insert at Beginning	**O(n)**	**O(1)**	Finds the last node to update its link.
2.Insert at End	**O(n)**	**O(1)**	Traverse to the last node.
3.Delete at Beginning	**O(n)**	**O(1)**	Finds the last node.
4.Delete at End	**O(n)**	**O(1)**	Traverse to the second-last node

5.Delete by value	**O(n)**	**O(1)**	Traverses to the specified position
6.Count Nodes	**O(n)**	**O(1)**	Visits every node
7.Display	**O(n)**	**O(1)**	Visits every node

LAB-5

5.aAIM: Write a programe of stack using array

Programme:

```
stack = []
def push():
    element = int(input("Enter element to push: "))
    stack.append(element)
    print(element, "pushed into stack")
def pop():
    if len(stack) == 0:
        print("Stack is empty")
    else:
        print(stack.pop(), "popped from stack")
def peek():
    if len(stack) == 0:
        print("Stack is empty")
    else:
```

```
        print("Top element is:", stack[-1])
def display():
    if len(stack) == 0:
        print("Stack is empty")
    else:
        print("Stack elements are:")
        for i in reversed(stack):
            print(i)
while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        push()
    elif choice == 2:
        pop()
    elif choice == 3:
        peek()
    elif choice == 4:
        display()
    elif choice == 5:
        print("Program ended")
        break
    else:
        print("Invalid choice")
```

output:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

================= RESTART: C:/Users/haisr/Downloads/DSA.5.1.py =================

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 2
2 pushed into stack

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 2
2 pushed into stack

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements are:
2
2

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
```

```
Enter "help" below or click "Help" above for more information.

================= RESTART: C:/Users/haisr/Downloads/DSA.5.1.py =================

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 2
2 pushed into stack

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 2
2 pushed into stack

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements are:
2
2

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program ended
```

Explanation:

- Stack is a linear data structure.it follows the LIFO (last in first out).
- Elements are stored in the array
- Push is used to insert an element. The element is added at the top. If the stack is full, Stack Overflow occurs.
- Pop is used to remove an element.
- The top element is removed.
- top is decreased. If the stack is empty, Stack Underflow occurs.
- Peek is used to view the top element.
- It does not remove the element.

- Display is used to show all elements.
- Elements are displayed from top to bottom.
- Error table:

Error	Rectified
stack.append()	stack.append(element)
stack.pop(0)	stack.pop()

5b.AIM: Stack using Linked list.

Programme:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
top = None
def push():
    global top

    element = int(input("Enter element to push: "))

    new_node = Node(element)
    new_node.next = top
    top = new_node

    print(element, "pushed into stack")
def pop():
    global top

    if top is None:
        print("Stack is empty")
    else:
        print(top.data, "popped from stack")
```

```
        top = top.next
def peek():
    if top is None:
        print("Stack is empty")
    else:
        print("Top element is:", top.data)
def display():
    if top is None:
        print("Stack is empty")
    else:
        temp = top
        print("Stack elements are:")

        while temp is not None:
            print(temp.data)
            temp = temp.next
while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        push()
    elif choice == 2:
        pop()
    elif choice == 3:
```

```
        peek()
    elif choice == 4:
        display()
    elif choice == 5:
        print("Program ended")
        break
    else:
        print("Invalid choice")
```

output:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

================= RESTART: C:/Users/haisr/Downloads/DSA.5.2.py =================

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 3
3 pushed into stack

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements are:
3

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements are:
3

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program ended
```

```
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

================= RESTART: C:/Users/haisr/Downloads/DSA.5.2.py =================

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 3
3 pushed into stack

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements are:
3

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements are:
3

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program ended
```

Explanation:

- A stack using linked list is a linear data structure that follows the LIFO (Last In, First Out) principle.
- In a linked-list stack, each element is stored in a node, and each node contains data and a link to the next node.
- A variable called top is used to point to the first node, which represents the top of the stack.
- Push operation is used to insert a new element. A new node is created and added at the top of the stack.

Pop operation is used to remove an element. The node pointed to by top is removed and top moves to the next node.

- Peek operation is used to view the top element without removing it from the stack.
- Display operation is used to traverse the linked list from top to None and display all the elements.
- A linked-list stack has dynamic size, so memory is allocated as needed. Stack Underflow occurs when we try to remove an element from an empty stack.

Error Table:

Error	Rectified
new.next = None	new_node.next = top
top = top	top = new_node

Lab-6

6a. AIM: Write a python program to implement queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach implement using array, linked list.

PROGRAM: Using array

```
class QueueEx:
    def __init__(self, size=5):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, x):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0

            self.rear += 1
            self.queue[self.rear] = x
            print(f"{x} inserted into the queue")
```

```
    def dequeue(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue Underflow")
        else:
            x = self.queue[self.front]
            self.queue[self.front] = None
            self.front += 1
            print(f"{x} deleted from the queue")

            if self.front > self.rear:
                self.front = -1
                self.rear = -1

    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            for i in range(self.front, self.rear + 1):
                print(self.queue[i])


size = int(input("Enter the size of Queue: "))
```

```
q = QueueEx(size)

while True:
    print("\n----- Queue Menu -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Exiting Program")
        break

    else:
```

```
        print("Invalid choice")
```

output:-



PROGRAM: Using linked list:

```
class QueueEx:
    def __init__(self, size=5):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1


    def enqueue(self, x):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0
```

```
        self.rear += 1
        self.queue[self.rear] = x
        print(f"{x} inserted into the queue")

    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        else:
            x = self.queue[self.front]
            self.queue[self.front] = None

            print(f"{x} deleted from the queue")
            self.front += 1

            if self.front > self.rear:
                self.front = -1
                self.rear = -1

    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            for i in range(self.front, self.rear + 1):
```

```
            print(self.queue[i])


size = int(input("Enter the size of Queue: "))
q = QueueEx(size)

while True:
    print("\n----- Queue Menu -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
```

```
            print("Exiting Program")
            break

        else:
            print("Invalid Choice!")
```

output:-

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

== RESTART: C:/Users/haisr/AppData/Local/Programs/Python/Python313/week6.2.py ==
Enter the size of Queue: 4

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 2
2 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 2

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Exiting Program
```

EXPLANATION:

· class QueueEx: Creates a class named QueueEx for implementing the queue.

· __init__(self, size=5): Constructor that initializes the queue with a default size of 5.

· self.size = size : Stores the user-given queue size.

· self.queue = [None] * size : Creates a list of the given size and initializes all positions with None.

· self.front = -1 : Initializes front to indicate that the queue is empty.

· self.rear = -1 : Initializes rear to indicate that no element is present.

enqueue()

· if self.rear == self.size - 1: Checks whether rear has reached the last position.

· self.front = 0 : Sets the front position when inserting the first element.

· self.rear += 1 : Increases rear to the next position.

· self.queue[self.rear] = x : Stores x at the rear position.

· print() : Displays the insertion message.

dequeue()

· if self.front == -1 or self.front > self.rear: Checks whether the queue is empty.

· x = self.queue[self.front] : Stores the front element in x.

· self.queue[self.front] = None : Removes the front element from the list.

· self.front += 1 : Moves front to the next position.

· if self.front > self.rear: : Checks whether all elements have been deleted.

· self.front = -1 and self.rear = -1 : Resets the queue to the empty state.

peek()

· if self.front == -1: Checks whether the queue is empty.

· self.queue[self.front] : Accesses and displays the front element without deleting it.

display()

· if self.front == -1: Checks whether the queue is empty.

· range(self.front, self.rear + 1) : Traverses the queue from front to rear.

· print(self.queue[i]) : Displays each queue element.

ERROR TABLE:

S.no	Errors	Rectified
1.	Forgetting the placing of semicolons	Rectified
2.	Indentation errors	Rectified

6b. AIM: Write a python program to implement queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach implement using circular linked list.

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularQueue:
    def __init__(self, size=5):
        self.size = size
        self.front = None
        self.rear = None
        self.count = 0

    def enqueue(self, x):
        if self.count == self.size:
            print("Queue Overflow")
        else:
            new_node = Node(x)

            if self.front is None:
                self.front = new_node
                self.rear = new_node
                new_node.next = new_node
            else:
                new_node.next = self.front
                self.rear.next = new_node
                self.rear = new_node

            self.count += 1
            print(f"{x} inserted into the queue")
```

```
def dequeue(self):
    if self.front is None:
        print("Queue Underflow")
    else:
        x = self.front.data

        if self.front == self.rear:
            self.front = None
            self.rear = None
        else:
            self.front = self.front.next
            self.rear.next = self.front

        self.count -= 1
        print(f"{x} deleted from the queue")

def peek(self):
    if self.front is None:
        print("Queue is empty")
    else:
        print("Front element:", self.front.data)

def display(self):
    if self.front is None:
        print("Queue is empty")
    else:
        print("The elements of the queue are:")

        temp = self.front
```

```
            while True:
                print(temp.data)
                temp = temp.next

                if temp == self.front:
                    break


size = int(input("Enter the size of Queue: "))
q = CircularQueue(size)

while True:
    print("\n----- Queue Menu -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()
```

```
    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Exiting Program")
        break

    else:
        print("Invalid Choice!")
```

OUTPUT:-

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

================== RESTART: C:/Users/haisr/Downloads/week6.3.py ==================
Enter the size of Queue: 3

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 1
1 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
The elements of the queue are:
1

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Exiting Program
```

EXPLANATION:

1. Node Linking

· self.next = None : Initializes the node link as empty.

· new_node = Node(x) : Creates a new node with the given data.

· new_node.next = new_node : Makes the first node point to itself, forming a circular link.

2. Circular Connection During Enqueue

· new_node.next = self.front : Connects the new node to the first node.

· self.rear.next = new_node : Connects the current last node to the new node.

· self.rear = new_node : Updates the last node to the newly inserted node.

3. Circular Connection During Dequeue

· self.front = self.front.next : Moves front to the next node.

· self.rear.next = self.front : Reconnects the last node to the new front node.

· if self.front == self.rear: : Checks whether only one node exists.

· self.front = None : Removes the only node by clearing the front reference.

· self.rear = None : Clears the rear reference.

4. Circular Traversal

· temp = self.front : Starts traversal from the first node.

· temp = temp.next : Moves to the next linked node.

· if temp == self.front: : Detects that traversal has returned to the first node.

· break : Stops traversal after one complete circular cycle.

5. Node Data Access

· self.front.data : Accesses the data stored in the front node.

· temp.data : Accesses the data of the current node during traversal.

ERROR TABLE:

Error table:

S.no	Errors	Rectification
1.	Indentation errors	Rectified
2.	Forgetting of placing semi colon.	Rectified

6c.AIM: Write a python programme to implement queue and its operations such as enqueue, dequeue, peek and display using Linked list data structure.

Programme:

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, x):
        if (self.rear + 1) % self.size == self.front:
            print("Queue overflow")
        else:
            if self.front == -1:
                self.front = 0
                self.rear = 0
            else:
                self.rear = (self.rear + 1) % self.size
            self.queue[self.rear] = x
            print(f"{x} inserted into the queue")

    def dequeue(self):
        if self.front == -1:
            print("Queue underflow")
        else:
            x = self.queue[self.front]
            self.queue[self.front] = None
            if self.front == self.rear:
                self.front = -1
                self.rear = -1
            else:
                self.front = (self.front + 1) % self.size
            print(f"{x} deleted from the queue")

    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
```

```
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            i = self.front
            while True:
                print(self.queue[i], end=" ")
                if i == self.rear:
                    break
                i = (i + 1) % self.size
            print()


if __name__ == "__main__":
    size = int(input("Enter the size of the circular queue: "))
    q = CircularQueue(size)

    while True:
        print("\n--- Circular Queue Operations Menu ---")
        print("1. Enqueue")
        print("2. Dequeue")
        print("3. Peek")
        print("4. Display")
        print("5. Exit")
        choice = input("Enter your choice (1-5): ")

        if choice == '1':
            val = int(input("Enter value to insert: "))
            q.enqueue(val)
        elif choice == '2':
            q.dequeue()
        elif choice == '3':
            q.peek()
        elif choice == '4':
            q.display()
        elif choice == '5':
```

```
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            i = self.front
            while True:
                print(self.queue[i], end=" ")
                if i == self.rear:
                    break
                i = (i + 1) % self.size
            print()


if __name__ == "__main__":
    size = int(input("Enter the size of the circular queue: "))
    q = CircularQueue(size)

    while True:
        print("\n--- Circular Queue Operations Menu ---")
        print("1. Enqueue")
        print("2. Dequeue")
        print("3. Peek")
        print("4. Display")
        print("5. Exit")
        choice = input("Enter your choice (1-5): ")

        if choice == '1':
            val = int(input("Enter value to insert: "))
            q.enqueue(val)
        elif choice == '2':
            q.dequeue()
        elif choice == '3':
            q.peek()
        elif choice == '4':
            q.display()
        elif choice == '5':
            print("Exiting program.")
            break
        else:
            print("Invalid choice! Please enter a number between 1 and 5.")
```

Output:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

================ RESTART: C:/Users/haisr/Downloads/week6.3.3.py ================
Enter the size of the circular queue: 3

--- Circular Queue Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 1
Enter value to insert: 2
2 inserted into the queue

--- Circular Queue Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 4
The elements of the queue are:
2

--- Circular Queue Operations Menu ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 5
```

Explanation:

- A circular queue uses a fixed-size list but connects the end back to the start, so space is reused.
- front and rear keep track of where elements are removed and added.
- When inserting, if (rear + 1) % size == front, the queue is full → overflow.
- If the queue is empty, both front and rear are set to 0 before inserting the first element.
- On enqueue, rear moves forward in a circular way using (rear + 1) % size.
- On dequeue, the element at front is removed, and front moves forward in a circular way.
- If after removing, front == rear, the queue becomes empty again (front = rear = -1).
- Display prints elements starting from front and looping until rear, showing the circular order.

Error table:

S.no	Errors	Rectified
1.	Indentation error	Rectified
2.	Forgetting to place front==rear	Rectified

WEEK-7

7.a.Aim: Write a python programme on binary tree using list and implement inorder, postorder, preorder, levelorder.

Programme:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BinaryTree:
```

```
def __init__(self):
    self.root = None

def create_tree(self):
    self.root = Node('A')
    self.root.left = Node('B')
    self.root.right = Node('C')
    self.root.left.left = Node('D')
    self.root.left.right = Node('E')
    self.root.right.right = Node('F')

def preorder(self, node):
    if node is None:
        return
    print(node.data, end=" ")
    self.preorder(node.left)
    self.preorder(node.right)

def inorder(self, node):
    if node is None:
        return
    self.inorder(node.left)
    print(node.data, end=" ")
    self.inorder(node.right)

def postorder(self, node):
    if node is None:
        return
    self.postorder(node.left)
    self.postorder(node.right)
```

```
        print(node.data, end=" ")

    def level_order(self):
        if self.root is None:
            return
        queue = [self.root]
        while queue:
            node = queue.pop(0)
            print(node.data, end=" ")
            if node.left is not None:
                queue.append(node.left)
            if node.right is not None:
                queue.append(node.right)


tree = BinaryTree()
tree.create_tree()

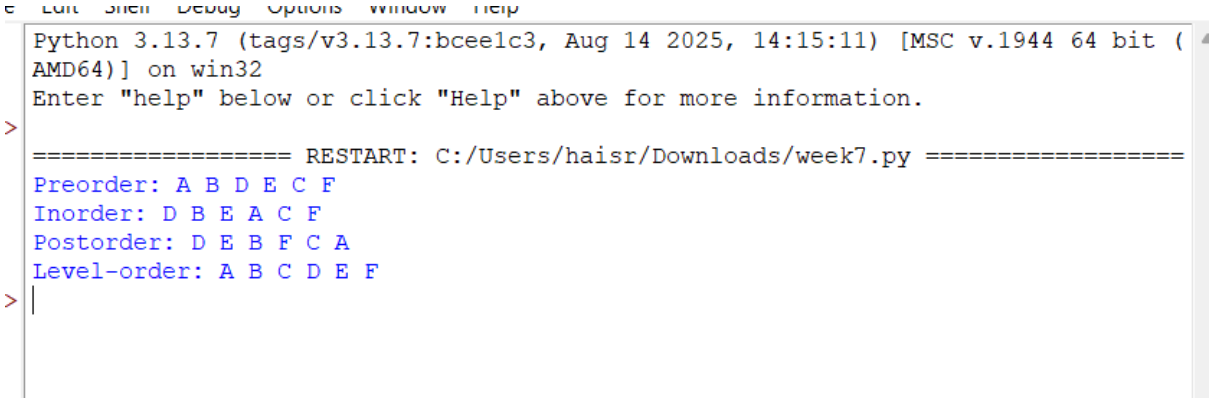
print("Preorder: ", end="")
tree.preorder(tree.root)

print("\nInorder: ", end="")
tree.inorder(tree.root)

print("\nPostorder: ", end="")
tree.postorder(tree.root)

print("\nLevel-order: ", end="")
tree.level_order()
```

Output:



Explanation:

- The Node class defines each tree element with data, left, and right links.
- The BinaryTree class manages the tree, starting with root = None.
- create_tree() builds a fixed binary tree with nodes A–F arranged in a specific structure.
- Recursive methods (preorder, inorder, postorder) traverse the tree in different orders and print node values.
- level_order() uses a queue to perform breadth-first traversal, visiting nodes level by level.
- The program prints traversal outputs: Preorder → A B D E C F, Inorder → D B E A C F, Postorder → D E B F C A, Level-order → A B C D E F.
- The recursive traversal functions stop when they reach a None node, ensuring the recursion terminates correctly.
- The queue in level_order() is a simple list, and nodes are processed in FIFO order to achieve breadth-first traversal.
- The print(node.data,end=" ") statements ensure all outputs appear on the same line with spaces instead of newlines.
- The tree structure is hardcoded in create_tree(); it doesn't take user input, so the same tree is always built.
- Each traversal method demonstrates a different way of exploring the same tree, useful for understanding tree algorithms.
- The program's final output shows how traversal orders differ, helping visualize the relationship between tree structure and traversal logic.

Error Table:

S.no	Errors	Rectified
1.	Instead of writing node written nod	Rectified

2.	Syntax error that is (:)	Rectified

7.bAim: Write a python programme on binary tree using array and implement inorder, preorder, postorder and levelorder.

Programme:

```
class BinaryTreeArray:
    def __init__(self, size):
        self.tree = [None] * size
        self.size = size

    def set_root(self, data):
        self.tree[0] = data

    def set_left(self, parent_index, data):
        child_index = 2 * parent_index + 1
        if child_index < self.size:
            self.tree[child_index] = data
        else:
            print("Index out of range!")

    def set_right(self, parent_index, data):
        child_index = 2 * parent_index + 2
        if child_index < self.size:
            self.tree[child_index] = data
        else:
            print("Index out of range!")
```

```
def preorder(self, index=0):
    if index >= self.size or self.tree[index] is None:
        return
    print(self.tree[index], end=" ")
    self.preorder(2 * index + 1)
    self.preorder(2 * index + 2)

def inorder(self, index=0):
    if index >= self.size or self.tree[index] is None:
        return
    self.inorder(2 * index + 1)
    print(self.tree[index], end=" ")
    self.inorder(2 * index + 2)

def postorder(self, index=0):
    if index >= self.size or self.tree[index] is None:
        return
    self.postorder(2 * index + 1)
    self.postorder(2 * index + 2)
    print(self.tree[index], end=" ")

def level_order(self):
    for value in self.tree:
        if value is not None:
            print(value, end=" ")

def display(self):
    print("\nArray Representation:")
    for i in range(self.size):
```

```
            print(f"Index {i}: {self.tree[i]}")


tree = BinaryTreeArray(7)

tree.set_root('A')
tree.set_left(0, 'B')
tree.set_right(0, 'C')
tree.set_left(1, 'D')
tree.set_right(1, 'E')
tree.set_right(2, 'F')

tree.display()

print("\nPreorder: ", end="")
tree.preorder()

print("\nInorder: ", end="")
tree.inorder()

print("\nPostorder: ", end="")
tree.postorder()

print("\nLevel-order: ", end="")
tree.level_order()
```

Output:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

================ RESTART: C:/Users/haisr/Downloads/week7.2..py ================

Array Representation:
Index 0: A
Index 1: B
Index 2: C
Index 3: D
Index 4: E
Index 5: None
Index 6: F

Preorder: A B D E C F
Inorder: D B E A C F
Postorder: D E B F C A
Level-order: A B C D E F
```

Explanation:

- The code creates a binary tree using an array instead of linked nodes.
- The root is stored at index 0, and children are calculated using formulas: left child → 2*parent_index+1, right child → 2*parent_index+2.
- If the calculated index is within the array size, the child is added; otherwise, it prints "Index out of range!".
- Preorder traversal prints the current node first, then left child, then right child.
- Inorder traversal prints the left child, then the current node, then the right child.
- Postorder traversal prints children first, then the current node.
- Level order traversal simply loops through the array and prints all non-None values.
- The display() method shows the array representation with index numbers and stored values.
- The tree is built with nodes A–F, and then all traversals are executed to show different ways of visiting nodes.
- This demonstrates how binary trees can be represented and traversed using arrays instead of node pointers.

Errortable:

S.no	Errors	Rectified
1.	Printing none values in level order	Rectified
2.	Root is not set before traversal	Rectified by calling set_root() first.

WEEK-8

8.aAim: Write a python programme to implement bineary search tree with its operation.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


def insert(root, data):
    if root is None:
        return Node(data)
    if data < root.data:
        root.left = insert(root.left, data)
    elif data > root.data:
        root.right = insert(root.right, data)
    return root


def search(root, key):
    if root is None or root.data == key:
        return root
    if key < root.data:
        return search(root.left, key)
    return search(root.right, key)


def inorder(root):
```

```
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)


def preorder(root):
    if root:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)


def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")


def delete(root, key):
    if root is None:
        return root

    if key < root.data:
        root.left = delete(root.left, key)
    elif key > root.data:
        root.right = delete(root.right, key)
    else:
        if root.left is None:
```

```
            return root.right
        if root.right is None:
            return root.left

        temp = root.right
        while temp.left:
            temp = temp.left

        root.data = temp.data
        root.right = delete(root.right, temp.data)

    return root


root = None

while True:
    print("\n1.Insert  2.Search  3.Delete")
    print("4.Inorder  5.Preorder  6.Postorder  7.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        x = int(input("Enter value: "))
        root = insert(root, x)

    elif ch == 2:
        x = int(input("Enter value to search: "))
        if search(root, x):
            print("Element found")
```

```
        else:
            print("Element not found")

    elif ch == 3:
        x = int(input("Enter value to delete: "))
        root = delete(root, x)

    elif ch == 4:
        inorder(root)

    elif ch == 5:
        preorder(root)

    elif ch == 6:
        postorder(root)

    elif ch == 7:
        break

    else:
        print("Invalid choice")
```

Output:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

================ RESTART: C:/Users/haisr/Downloads/week8.1.py ================

1.Insert  2.Search  3.Delete
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 1
Enter value: 2

1.Insert  2.Search  3.Delete
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 2
Enter value to search: 2
Element found

1.Insert  2.Search  3.Delete
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 7
```

Explanation:

1. The Node class is used to create a node containing data and two pointers, left and right, which point to the left and right child nodes.

2. The insert() function adds a new node to the Binary Search Tree. Smaller values are placed on the left side, and larger values are placed on the right side.

3. The search() function checks whether the given element exists in the tree. It compares the element with the current node and searches the left or right subtree.

4. The delete() function removes a node from the tree. It handles nodes with no children, one child, or two children while maintaining the BST property.

5. The inorder(), preorder(), and postorder() functions display the tree elements in different orders. Inorder displays elements in ascending order.

6. The while loop displays a menu and allows the user to choose operations such as insert, search, delete, and traversal. The program continues until the user selects Exit.

Error table:

S.no	Errors	Rectified
1.	Using an undefined variable or function name	Rectified
2.	Passing a value of the wrong data type.	Rectified.

WEEK-9

9.aAim: Write a python program to implement avl trees with its operation.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
        self.height = 1

def height(root):
    if root is None:
        return 0
    return root.height

def balance(root):
    if root is None:
        return 0
    return height(root.left) - height(root.right)

def rightRotate(y):
    x = y.left
    t = x.right

    x.right = y
    y.left = t

    y.height = 1 + max(height(y.left), height(y.right))
    x.height = 1 + max(height(x.left), height(x.right))
    return x

def leftRotate(x):
    y = x.right
    t = y.left

    y.left = x
    x.right = t

    x.height = 1 + max(height(x.left), height(x.right))
    y.height = 1 + max(height(y.left), height(y.right))
    return y

def insert(root, data):
```

```
    x.height = 1 + max(height(x.left), height(x.right))
    y.height = 1 + max(height(y.left), height(y.right))
    return y

def insert(root, data):
    if root is None:
        return Node(data)

    if data < root.data:
        root.left = insert(root.left, data)
    elif data > root.data:
        root.right = insert(root.right, data)
    else:
        return root

    root.height = 1 + max(height(root.left), height(root.right))
    b = balance(root)

    if b > 1 and data < root.left.data:
        return rightRotate(root)

    if b < -1 and data > root.right.data:
        return leftRotate(root)

    if b > 1 and data > root.left.data:
        root.left = leftRotate(root.left)
        return rightRotate(root)

    if b < -1 and data < root.right.data:
        root.right = rightRotate(root.right)
        return leftRotate(root)

    return root

def search(root, key):
    if root is None or root.data == key:
        return root
    if key < root.data:
        return search(root.left, key)
    return search(root.right, key)
```

```
    if root is None or root.data == key:
        return root
    if key < root.data:
        return search(root.left, key)
    return search(root.right, key)

def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)

def minNode(root):
    while root.left:
        root = root.left
    return root

def delete(root, key):
    if root is None:
        return root

    if key < root.data:
        root.left = delete(root.left, key)
    elif key > root.data:
        root.right = delete(root.right, key)
    else:
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left

        temp = minNode(root.right)
        root.data = temp.data
        root.right = delete(root.right, temp.data)

    if root is None:
        return root

    root.height = 1 + max(height(root.left), height(root.right))
    b = balance(root)
```

```
        root.left = leftRotate(root.left)
        return rightRotate(root)

    if b < -1 and balance(root.right) <= 0:
        return leftRotate(root)

    if b < -1 and balance(root.right) > 0:
        root.right = rightRotate(root.right)
        return leftRotate(root)

    return root

oot = None

hile True:
    print("\n1.Insert  2.Delete  3.Search")
    print("4.Inorder  5.Exit")
    ch = int(input("Enter choice: "))

    if ch == 1:
        x = int(input("Enter value: "))
        root = insert(root, x)
    elif ch == 2:
        x = int(input("Enter value to delete: "))
        root = delete(root, x)
    elif ch == 3:
        x = int(input("Enter value to search: "))
        if search(root, x):
            print("Element found")
        else:
            print("Element not found")
    elif ch == 4:
        inorder(root)
        print()
    elif ch == 5:
        break
    else:
        print("Invalid choice")
```

Output:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

================ RESTART: C:/Users/haisr/Downloads/week9.1.py ================

1.Insert  2.Delete  3.Search
4.Inorder  5.Exit
Enter choice: 1
Enter value: 2

1.Insert  2.Delete  3.Search
4.Inorder  5.Exit
Enter choice: 3
Enter value to search: 2
Element found

1.Insert  2.Delete  3.Search
4.Inorder  5.Exit
Enter choice: 4
2

1.Insert  2.Delete  3.Search
4.Inorder  5.Exit
Enter choice: 5
```

Explanation:

- The Node class creates a node with data, left and right pointers, and height.
- The height() and balance() functions calculate the height and balance factor of each node.
- The leftRotate() and rightRotate() functions balance the tree when it becomes unbalanced.
- The insert() function inserts a new element and performs rotations when necessary to maintain AVL balance.
- The delete() function removes an element and balances the tree again. The search() function checks whether an element exists.
- The inorder() function displays the elements in ascending order, while the menu allows the user to select different operations.

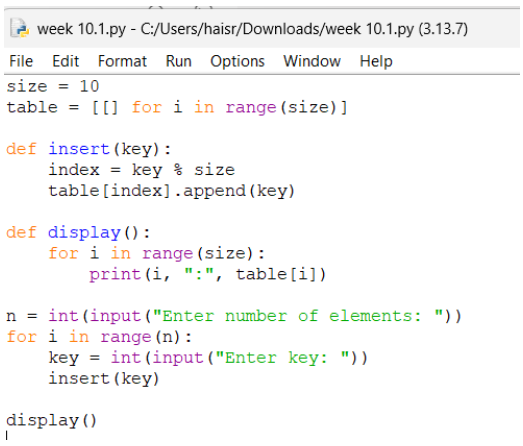
Error table:

S.no	Errors	Rectified
1.	Using an undefined variable or function.	Rectified
2.	Entering a non-integer value for the menu choice.	Rectified.

WEEK-10

10.aAim: Python programs for hashing using separate chaining, linear probing, and quadratic probing.

Program:



Output:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

================ RESTART: C:/Users/haisr/Downloads/week 10.1.py ================
Enter number of elements: 2
Enter key: 2
Enter key: 2
0 : []
1 : []
2 : [2, 2]
3 : []
4 : []
5 : []
6 : []
7 : []
8 : []
9 : []
```

Explanation:

- The size = 10 statement creates a hash table with 10 positions.
- The table = [[] for i in range(size)] statement creates an empty list at every position.
- The insert() function calculates the index using key % size.
- If two keys have the same index, both keys are stored in the same list.
- The display() function prints each index and its stored elements.
- The for loop accepts keys from the user and inserts them into the hash table.

Error table:

S.no	Errors	Rectified
1.	Adding incompatible data types.	Rectified
2.	Entering a non-integer key.	Rectified.

10.bProgram: Hashing using linear probing.

week10.2.py - C:/Users/haisr/Downloads/week10.2.py (3.13.7)

File Edit Format Run Options Window Help

```
size = 10
table = [-1] * size

def insert(key):
    index = key % size

    while table[index] != -1:
        index = (index + 1) % size

    table[index] = key

def display():
    for i in range(size):
        print(i, ":", table[i])

n = int(input("Enter number of elements: "))
for i in range(n):
    key = int(input("Enter key: "))
    insert(key)

display()
```

Output:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

================ RESTART: C:/Users/haisr/Downloads/week10.2.py ================
Enter number of elements: 3
Enter key: 2
Enter key: 1
Enter key: 2
0 : -1
1 : 1
2 : 2
3 : 2
4 : -1
5 : -1
6 : -1
7 : -1
8 : -1
9 : -1
```

Explanation:

- The size = 10 statement defines the size of the hash table.
- The table = [-1] * size statement creates an empty table, where -1 represents an empty position.
- The insert() function calculates the index using key % size.
- If the position is occupied, the while loop checks the next position using (index + 1) % size.
- The display() function prints all indices and their values.
- The for loop accepts keys from the user and inserts them into the hash table using linear probing.

Error table:

S.no	Errors	Rectified
1.	Using an undefined variable or function.	Rectified
2.	Entering invalid input.	Rectified.

10.cProgram: Hashing using quadratic probing.

```
size = 10
table = [-1] * size

def insert(key):
    index = key % size
    i = 0

    while i < size:
        new_index = (index + i * i) % size

        if table[new_index] == -1:
            table[new_index] = key
            return

        i += 1

    print("Hash table is full or no slot found")

def display():
    for i in range(size):
        print(i, ":", table[i])

n = int(input("Enter number of elements: "))
for i in range(n):
    key = int(input("Enter key: "))
    insert(key)

display()
```

Output:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

================ RESTART: C:/Users/haisr/Downloads/week10.3.py ================
Enter number of elements: 3
Enter key: 1
Enter key: 2
Enter key: 3
0 : -1
1 : 1
2 : 2
3 : 3
4 : -1
5 : -1
6 : -1
7 : -1
8 : -1
9 : -1
```

Explanation:

- The size = 10 statement defines the size of the hash table.
- The table = [-1] * size statement creates an empty table, where -1 indicates an empty position.
- The insert() function calculates the initial index using key % size.
- The while loop checks positions using (index + i*i) % size when a collision occurs.
- If an empty position is found, the key is inserted there; otherwise, the loop continues checking positions up to the table size.
- The display() function prints the hash table, while the for loop accepts keys from the user and inserts them.

Error table:

S.no	Errors	Rectified
1.	Using an undefined variable or function.	Rectified
2.	Keys are inserted successfully.	Rectified.

Time Complexity And Space Complexity:

Hashing Technique	Best Time	Average Time	Worst Time	Space Complexity
Separate Chaining	O(1)	O(1)	O(n)	O(n+m)
Linear Probing	O(1)	O(1)	O(n)	O(m)
Quadratic Probing	O(1)	O(1)	O(n)	O(m)